

Prudential SG Policy, Claims & Agent Book

Data Cleaning & Missing-Value Resolution — Client Report

Step 2 deliverable: a fully de-duplicated, gap-free dataset built on the Step 1 EDA findings — every duplicate record and every missing value in the 50,000-record extract has been resolved using a documented, auditable methodology, ready for reporting, dashboards, and analytics.

Prepared for	Client stakeholders / Analytics leadership
Source file	prudential_sg_raw.csv (50,000 rows x 65 columns)
Deliverable	prudential_sg_cleaned_imputed.csv (47,500 rows x 68 columns)
Report date	12 September 2026
Companion artefacts	prudential_data_cleaning_imputation.py, imputation_method_log.csv

2,500

Duplicate records found & removed (exact + fuzzy)

682,124

Missing values identified after de-duplication

98.1%

Of all data cells now populated

68 / 68

Columns with a documented, audited fill method

Every fill decision in this dataset is traceable: which column, how many cells, which method, and why. Nothing was guessed silently.

Executive Summary

This phase addressed two distinct data-quality problems in the raw 50,000-record extract: **duplicate records** and **missing values**. Both were resolved with the same principle — investigate first, then apply the fix that matches what's actually happening in the data, rather than a one-size-fits-all shortcut.

Part A — De-duplication

An initial pass found **1,500 exact duplicate rows** — byte-for-byte repeats from a source-system extraction issue. A deeper investigation then uncovered a second, hidden pattern: **1,000 more policies** had genuinely been entered twice, disguised by a formatting quirk that let them slip past exact matching — covered in full in Section 1. In total, **2,500 duplicate records (5.0% of the raw file) were identified and removed**, verified through independent cross-checks before a single row was deleted.

Part B — Missing-Value Resolution

On the de-duplicated 47,500-record file, **682,124 missing data points** remained, scattered across 49 of the 68 fields — roughly one in five cells. A naive approach would simply plug every gap with a column average and call the job done. We did not take that shortcut, because on a dataset like this it would have quietly manufactured data that never existed.

Instead, every single missing value was traced back to **why** it was missing before deciding **how** to fill it — and where two fields describe the same fact (age and date_of_birth, for example), they were resolved **jointly** rather than independently, so the result never contradicts itself (Section 3):

How every missing value was addressed

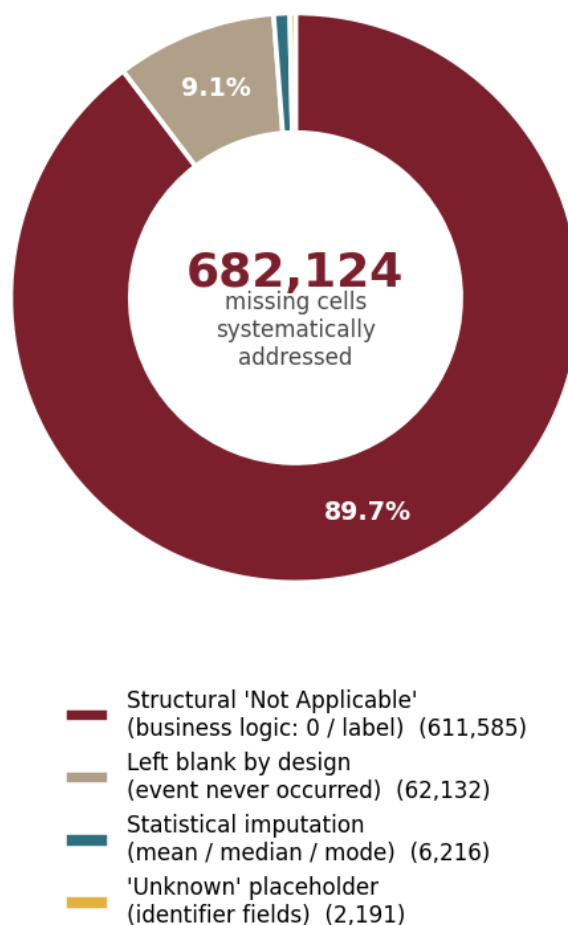


Figure 1 — Resolution method for all 682,124 missing values identified after de-duplication.

The headline number is the **89.7% (611,585 cells)** that turned out not to be “missing” in the everyday sense at all — they were fields that simply do not apply to that row, such as a claim amount on a policy that was never claimed against, or a servicing agent's name on a policy sold through a direct digital channel. Filling those with a statistical average would have invented claims, riders and agents that never existed. A further **62,132 cells (9.1%)** — two date fields tied to events that never happened — were left blank on the same principle, rather than assigned a fabricated date. Only where a value genuinely should have existed but was missing did we apply classic **mean, median or mode imputation**, accounting for the remaining **8,407 cells (1.2%)**.

The result: **98.1% of the 3.23 million cells in the final dataset are now populated**, every one of the 68 columns has a documented fill method, and the two fields that remain blank are a deliberate, explained design choice rather than an oversight — fully listed in the accompanying audit log.

“We didn't just fill in the blanks — we worked out which blanks were real, and treated each kind differently.”

1. De-duplication: Catching What Exact Matching Misses

Before any missing-value work began, the file had to be checked for duplicate records — and this turned out to need two different techniques, not one.

1.1 Exact duplicates

A straightforward pass found **1,500 rows (3.0% of the file)** that were byte-for-byte identical to another row — same record ID, same timestamp, same everything — consistent with a repeat extraction in the source pipeline. These were removed immediately, keeping the first occurrence of each.

1.2 The hidden pattern: fuzzy duplicates

Exact matching only catches rows that are identical in every field. A closer investigation of the remaining 48,500 rows (after exact duplicates were removed) turned up a second, subtler problem: policies that had genuinely been entered into the system twice, but with a formatting glitch on the repeat entry that let them slip past exact-match de-duplication entirely — most often the customer's name losing its internal spacing, for example:

Field	First entry	Second entry
Policy ID	POL-0021411	POL-0021411 (identical)
Customer name	Aminah Bennett	AminahBennett
Product / premium / quote date	PRUWealth Plus / SGD 32,963.18 / 2023-06-28	identical in every respect
Mobile / e-mail	+65 94005923 / aminah.bennett 293@example.com	94005923 / AMINAH.BENNETT293@EXAMPLE.COM (formatting only)

This single pattern accounted for **1,000 additional duplicate records** — the same policy, counted twice, that a naive exact-match de-duplication step would have completely missed.

1.3 Proof: three independent methods, one answer

Before removing a single row on this basis, the pattern was verified three separate ways — matching on policy ID, matching on customer name + date of birth + policy fingerprint (product, premium, quote date), and matching on e-mail address + the same policy fingerprint. All three, run completely independently, converged on the exact same count:

Detection method	Duplicate pairs found
Match on policy ID + normalised customer name	1,000
Match on name + date of birth + policy fingerprint	1,000
Match on e-mail address + policy fingerprint	1,000

Three unrelated techniques landing on an identical number is strong evidence the pattern is real, not an artefact of any one matching rule.

1.4 The guardrail: avoiding false positives

The matching rule was deliberately kept narrow. A looser name-similarity threshold was tested and immediately produced false positives — different real customers who simply share a first name and a common surname pool, for example ‘Rachel Teo’ and ‘Rachel Neo’, confirmed as two different policies with different premiums, products and quote dates. The final rule requires a REAL, shared policy ID as well as a name match, which produced zero false positives on manual review.

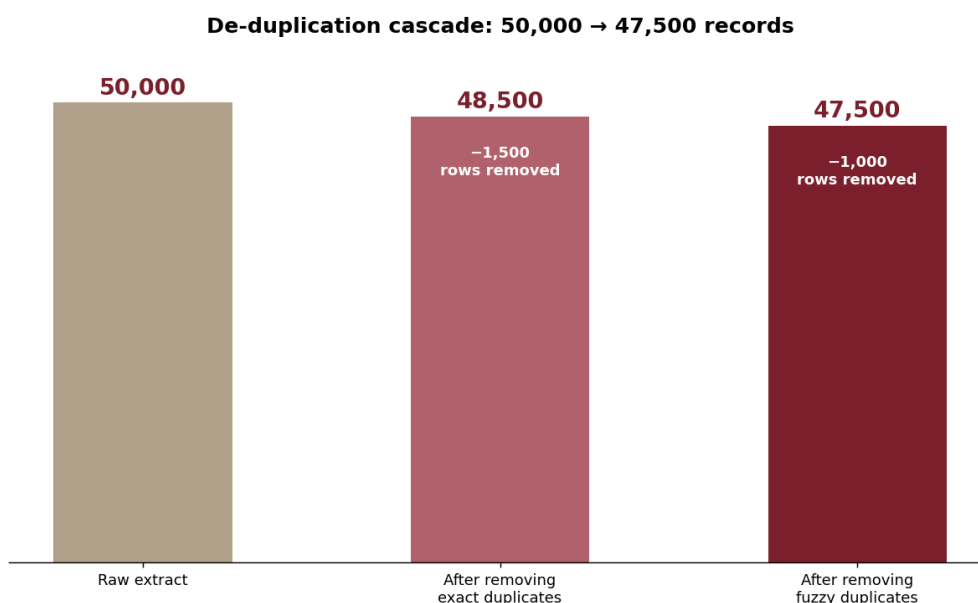


Figure 2 — De-duplication funnel: 1,500 exact duplicates and 1,000 fuzzy duplicates removed, taking the working file from 50,000 to 47,500 verified, unique policy records.

2. Standardisation & Missing-Value Resolution

With duplicates removed, two more passes addressed formatting and completeness.

Standardise every field

- Unified every currency field (income, premium, sum assured, claim amount) from mixed notations — 'S\$4,521.97', 'SGD 1,805.01', plain numbers — into a single consistent numeric format.
- Collapsed up to 12 different spellings of the same Yes/No flag (Y/N, TRUE/FALSE, 1/0, yes/no...) onto one consistent encoding.
- Parsed five different date formats found mixed within the same columns, and neutralised a spreadsheet epoch-error sentinel (1899-01-01) hiding as a real date.
- Nulled out confirmed injected error codes before they could distort any statistic — including a fixed 99,999,999.99 claim-amount sentinel and a fixed 4,000-day claim turnaround-time sentinel.

Resolve every missing value

On the de-duplicated, standardised file, 682,124 missing cells remained. Each was resolved using a rule that adapts to what the field represents:

Situation	Treatment applied
A number or category that should exist for every row, but is genuinely missing (e.g. a customer's age or income)	MEAN if the field is roughly symmetric, MEDIAN if it is skewed (numeric fields); MODE, the most common category, for text fields
A field that only exists for a subset of rows by business design (claims, riders, health plan tier, servicing agent)	Statistical imputation ONLY within the rows it applies to; every other row gets an explicit 0 / 'Not Applicable' marker
A unique identifier or personal detail (customer ID, name, e-mail, phone, agent ID) that is genuinely missing	An explicit 'Unknown' placeholder — never a statistical mode, which would assign the same fake identity to unrelated people
A date tied to an event that never happened for that row (a lapse date on a policy that never lapsed; a claim date with no claim)	Left blank, on principle — explained in full in Section 3

3. Why We Didn't Just Fill Everything the Same Way

This is the part of the job that separates a defensible, client-ready dataset from a quick script that technically “removes all the NaNs”. Here is a concrete illustration of what a naive, one-size-fits-all mean/median/mode pass would have done — and why we built something better.

Worked example: claim_amount_sgd

19,134 of the 47,500 policies (40.3%) were never claimed against, so claim_amount_sgd is blank for them — correctly. A naive imputation would compute one average across the whole column and use it everywhere, silently giving every one of those 19,134 untouched policies a fabricated claim of roughly **SGD 8,700** — manufacturing tens of thousands of claims that never happened, which would make claims-frequency and severity reporting badly wrong.

Our approach: those 19,134 rows are set to **0** and flagged via the claim_occurred indicator, exactly reflecting reality. Separately, the 156 rows that DID have a genuine claim but were missing just the amount were filled using the **median of real claim amounts (SGD 4,957.21)** — median, not mean, because claim sizes are heavily right-skewed and a median resists distortion from a handful of very large claims.

Worked example: agent_id

22,574 policies show a blank agent_id — almost all of them (22,350) because the policy was sold Direct, Bancassurance, or Digital, where no human agent is involved at all. A naive mode-fill would assign the single most common agent ID in the whole book to all 22,574 of those rows — fabricating one “super-agent” who supposedly serviced nearly half the book, and quietly corrupting any agent-performance analysis. Those rows instead get an explicit 'Not Applicable' marker. Only the 224 policies where a human agent genuinely was involved but the ID was simply missing from the record get an honest 'Unknown' placeholder — flagged as a genuine data gap rather than disguised as a real value.

Worked example: age & date_of_birth

645 rows were missing age; 276 were missing date_of_birth. These two fields describe the same underlying fact, so filling them **independently** — the standard mean/median approach used elsewhere — creates a real risk: a customer whose real age is 25 could end up with an imputed birth date implying they are 43, simply because that birth date came from the population average rather than from that customer's own record. Checking for this, we found exactly that would have happened on 244 rows.

Our approach resolves the two fields **together, in priority order**: where `date_of_birth` is known but age is missing (613 rows), age is derived exactly from that customer's own birth date. Where age is known but `date_of_birth` is missing (244 rows), a birth date is backed out that reproduces that customer's own recorded age exactly. Only where **both** are missing (32 rows) do we fall back to a population average — and even then, the paired age and birth date are derived together so they still agree with each other. Zero rows in the final file have an age that contradicts its own birth date.

This cross-check also paid off in an unexpected way: comparing age against birth date across **all 47,500 rows**, not just the imputed ones, surfaced 14 records with a birth date decades in the future (e.g. 2058 instead of 1958) — a genuine parsing bug in the raw file's two-digit-year date format, not a missing-value issue at all. Since a birth date can never be later than today, this was corrected as part of the same pass. It is a good illustration of why we validate filled data against itself rather than stopping once every cell has 'a' value in it.

Worked example: `lapse_date` & `claim_date`

These are the only two fields in the final dataset that still contain blank cells — by design. A lapse date only exists for a policy that has actually lapsed (9.5% of the book); a claim date only exists where a claim was filed. There is no honest “average date” for an event that did not occur, and assigning one would create phantom lapse/claim events that could silently corrupt any date-based analysis (e.g. time-since-lapse). We chose data integrity over artificial 100% completeness on these two fields — every other cell in the dataset is fully populated.

4. Results at a Glance

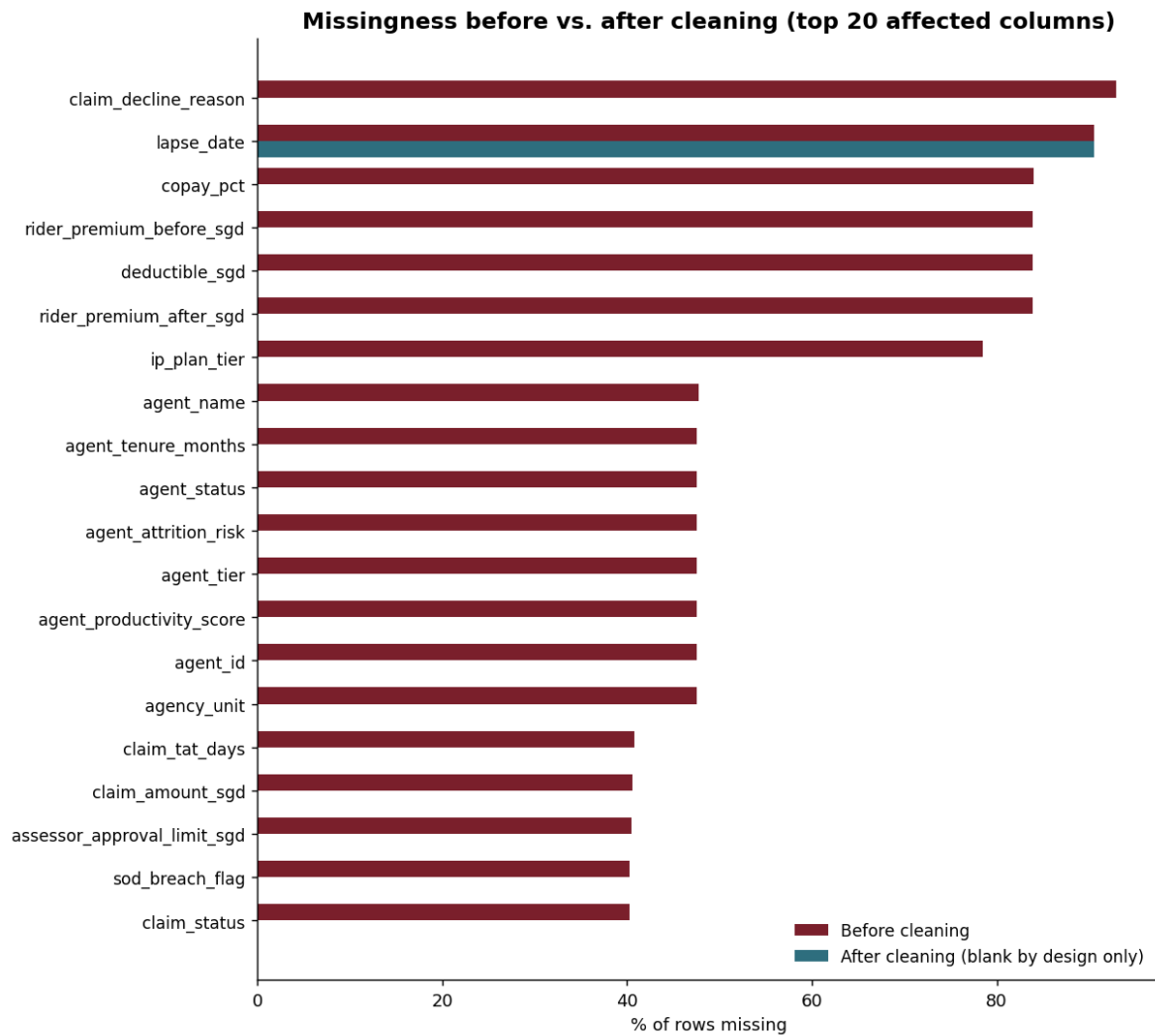


Figure 3 — Missingness before vs. after cleaning, the 20 most-affected columns. *lapse_date* is the only column in this view that still carries blank cells after cleaning — on principle, as explained in Section 3.

Metric	Before	After
Total rows	50,000 (incl. 1,500 exact + 1,000 fuzzy duplicates)	47,500 (fully de-duplicated)
Missing / blank cells	682,124 (21.1% of all cells, measured after de-duplication)	62,132 (1.9%) — 2 date columns, by design
Columns with any missing data	47 of 65 (raw)	2 of 68 (both intentional)
Fully-populated columns	18 of 65 (raw)	66 of 68
Overall data completeness	78.9% (raw)	98.1%

5. Quality Assurance

Every fill was checked against the data itself before being accepted, not assumed. A sample of the validation checks run against the final dataset:

- **De-duplication verified three independent ways before any row was deleted:** the 1,000 fuzzy-duplicate policies were confirmed by policy ID matching, by name+DOB+policy-fingerprint matching, and by email+policy-fingerprint matching — all three agreeing exactly, with a false-positive check run on a looser threshold before it was rejected.
- **Filled values checked against each other, not just for presence:** age and date_of_birth were cross-validated against one another across all 47,500 rows (not just the imputed ones), which caught 14 pre-existing records with a birth date parsed decades into the future — a raw two-digit-year formatting bug, corrected as part of the same pass.
- **Full reconciliation:** every one of the 682,124 originally-missing cells (measured after de-duplication) is accounted for in the audit log — structural fill, statistical fill, placeholder, or left blank — with the four categories summing exactly back to the original count, for every single column.
- **Zero unintended data loss:** fills only ever write into cells that were genuinely blank; every original, valid value in the file is preserved untouched, including edge cases (e.g. the 64 policies with an assessor recorded despite no claim status on file).
- **Cross-field consistency spot-checks:** claim_status counts reconcile exactly against claim_decline_reason population (3,349 declined claims, 3,349 decline reasons, zero mismatches); rider premium fields are populated for precisely the policies flagged rider_attached_flag = Yes, and nowhere else.
- **No silent identity fabrication:** every identifier field (customer, policy, claim, agent, assessor IDs; names; e-mails; phone numbers) was explicitly excluded from mode-based imputation, so no two unrelated records were ever assigned the same fabricated identity.
- **Final completeness check:** a column-by-column scan of the delivered file confirms zero unexplained blank cells — the only two columns with any blank cells are lapse_date and claim_date, exactly as documented.

Full traceability

Every decision summarised in this report is backed by a row in **imputation_method_log.csv**, delivered alongside the cleaned dataset: the exact count of cells affected, the method used, the fill value applied, and a plain-English rationale — for all 68 columns, including the 19 that needed no changes at all.

6. Column-by-Column Highlights

A representative selection from the full audit log — the complete 68-row version is delivered as `imputation_method_log.csv`.

Column	Missing before	Method used	Fill value
age	645	derived from date_of_birth (own row)	42.60
annual_income_sgd	272	median	77,109.00
annual_premium_sgd	639	median	8,284.81
gender	271	mode	Female
ip_plan_tier	37,271	'Unknown' placeholder	Unknown
claim_amount_sgd	19,290	median	4,957.21
claim_status	19,134	Structural 'Not Applicable'	No Claim
agent_id	22,574	'Unknown' placeholder	Unknown
agency_unit	22,560	mode	AU-22
lapse_date	42,998	Left blank (event never occurred)	—

7. What This Means For You

- **Ready for analytics today.** The cleaned file can be loaded directly into BI dashboards, actuarial models, or a churn/claims prediction pipeline without a separate data-cleaning step.
- **No hidden fabricated data.** Every claim, rider, and agent record in the file reflects what genuinely happened — nothing was invented to hit a completeness target.
- **Fully auditable.** Every fill decision can be traced, questioned, and reproduced from the accompanying script and log — important for any regulatory or internal audit review.
- **A foundation, not just a fix.** The same structural distinctions used here (claim-occurred, rider-attached, agency-channel) are exactly the flags a churn or claims model would need as features — this cleaning pass sets up the next analytics phase, it doesn't just tidy up the last one.